

React Hook: useScope — Efficient Information Scope Management

Optimized for FastAPI/Obsidian/Dola Stack •
Isolate State • Reduce Re-renders • Clean Context

Core Concept

Information Scope = Isolated Data Boundary

- Separate global/app/module/component scopes
- Auto-cleanup on unmount/scope change
- Memoized → only updates when relevant
- No prop drilling + lightweight alternative to full state managers



Full Source: [src/hooks/useScope.js](#)

jsx

```
import { useMemo, useRef, useEffect, useState,
useCallback } from 'react';
```

```
// 🔑 Scope Registry (shared across app)
```

```
const SCOPE_REGISTRY = new Map();
```

```
const SCOPE_EVENTS = new EventTarget();
```

```
/**
```

- useScope — Efficient Information Scope Manager
- @param {string} scopeld - Unique ID: 'app', 'auth', 'obsidian', 'dola:pr', etc.
- @param {object} initialData - Initial scope data
- @param {object} options - { persist: boolean, isolate: boolean, deps: any[] }
- @returns {object} { data, setData, update,

```
reset, subscribe, isActive }
```

```
*/
```

```
export function useScope(scopeld, initialData  
= {}, options = {}) {
```

```
  const { persist = false, isolate = true, deps = []  
} = options;
```

```
  const scopeRef = useRef({ id: scopeld, data:  
initialData, listeners: new Set() });
```

```
  const [, forceRender] = useState(0);
```

```
// 🧱 Initialize / Register Scope
```

```
useMemo(() => {
```

```
  if (!SCOPE_REGISTRY.has(scopeld)) {
```

```
    SCOPE_REGISTRY.set(scopeld, {
```

```
      data: initialData,
```

```
      createdAt: Date.now(),
```

```
      accessCount: 0
```

```
    });
```

```
  }
```

```
  scopeRef.current.data =
```

```
    SCOPE_REGISTRY.get(scopeld).data;
```

```
}, [scopeld, ...deps]);
```

```
//  Memoized Update (Shallow Diff)
```

```
const setData = useCallback((newData) => {  
  const current =  
    SCOPE_REGISTRY.get(scopeld)?.data || {};  
  const next = typeof newData === 'function' ?  
    newData(current) : newData;
```

```
//  Skip if identical → Prevent re-render  
if (isShallowEqual(current, next)) return;
```

```
//  Update & Notify
```

```
if (SCOPE_REGISTRY.has(scopeld)) {  
  SCOPE_REGISTRY.get(scopeld).data = {  
    ...current, ...next };  
  scopeRef.current.data =  
    SCOPE_REGISTRY.get(scopeld).data;  
  SCOPE_EVENTS.dispatchEvent(new  
    CustomEvent(scope:${scopeld}`, { detail:  
      scopeRef.current.data }));  
  forceRender(n => n + 1);
```

```
}
```

```
}, [scopeId]);
```

```
// 📌 Partial Update (Patch)
```

```
const update = useCallback((patch) => {  
  setData(prev => ({ ...prev, ...patch }));  
}, [setData]);
```

```
// 🔄 Reset to Initial
```

```
const reset = useCallback(() => {  
  setData(initialData);  
}, [scopeId, initialData]);
```

```
// 📡 Subscribe to Scope Changes (Cross-  
Component)
```

```
const subscribe = useCallback((callback) => {  
  const handler = e => callback(e.detail);  
  SCOPE_EVENTS.addEventListener(scope:$  
    {scopeId}, handler); return () => {  
    SCOPE_EVENTS.removeEventListener(scope:$  
      {scopeId}, handler);  
  };  
}, [scopeId]);
```

```
// 🧹 Cleanup: Auto-remove if NOT persistent
useEffect(() => {
  return () => {
    if (!persist && !isolate) {
      SCOPE_REGISTRY.delete(scopeld);
      SCOPE_EVENTS.dispatchEvent(new
        CustomEvent(scope:${scopeld}:clear`));
    }
  };
}, [scopeld, persist, isolate]);
```

```
// 📊 Scope Metadata
const meta = useMemo(() => ({
  isActive: SCOPE_REGISTRY.has(scopeld),
  createdAt:
    SCOPE_REGISTRY.get(scopeld)?.createdAt,
  accessCount: +
    +SCOPE_REGISTRY.get(scopeld).accessCount
}), [scopeld]);
```

```
return useMemo(() => ({
```

```
scopeld,  
data: scopeRef.current.data,  
setData,  
update,  
reset,  
subscribe,  
...meta  
)), [scopeld, setData, update, reset, subscribe,  
meta]);  
}
```

//  Helper: Shallow Equality Check

```
function isShallowEqual(a, b) {  
  if (a === b) return true;  
  if (typeof a !== 'object' || a === null || typeof b !==  
  'object' || b === null) return false;  
  const keysA = Object.keys(a);  
  const keysB = Object.keys(b);  
  if (keysA.length !== keysB.length) return false;  
  for (let key of keysA) if (a[key] !== b[key]) return  
  false;  
  return true;  
}
```

```
}
```

```
// 📁 Bulk Scope Selector (Optimize Multiple  
Scopes)
```

```
export function useScopes(scopelds = []) {  
  return useMemo(() => ({  
    getAll: () => scopelds.map(id =>  
      SCOPE_REGISTRY.get(id)?.data),  
    get: (id) => SCOPE_REGISTRY.get(id)?.data,  
    has: (id) => SCOPE_REGISTRY.has(id)  
  }), [scopelds.join('|')]);  
}
```

```
// 🧹 Utility: Clear All Scopes
```

```
export function clearAllScopes() {  
  SCOPE_REGISTRY.clear();  
  SCOPE_EVENTS.dispatchEvent(new  
    CustomEvent('scopes:cleared'));  
}
```

Usage Examples — Full Stack (Dola/Obsidian/FastAPI)

1 App-Wide Scope: src/scopes/AppScope.jsx

jsx

```
import { useScope } from '../hooks/useScope';
```

```
export function useAppScope() {
```

```
  return useScope('app', {
```

```
    theme: 'dark',
```

```
    apiUrl: 'http://localhost:8000',
```

```
    obsidianUrl: 'http://localhost:27124',
```

```
    isOnline: true
```

```
  }, { persist: true }); //  Persist across mounts
```

```
}
```

2 Auth Scope (Isolated): src/scopes/AuthScope.jsx

```
jsx
export function useAuthScope() {
  return useScope('auth', {
    user: null,
    token: null,
    isAuthenticated: false,
    permissions: []
  }, { isolate: true }); //  No bleed to other scopes
}
```

3 Dola AI PR Scope (Component-Specific):

```
jsx
function PRReviewCard({ prId }) {
  //  Unique scope per PR → No conflict
  const { data, update, subscribe } =
    useScope(dola:pr:${prId}`, {
      status: 'pending',
      checks: [],
      comments: [],
```

```
lastUpdated: null
```

```
});
```

```
//  Auto-update when scope changes
```

```
useEffect(() => subscribe(update), [subscribe,  
update]);
```

```
return (
```

```
<div className="card">
```

```
<h3>PR #{prId} — {data.status}</h3>
```

```
<pre>{JSON.stringify(data.checks, null, 2)}</pre>
```

```
<button onClick={() => update({ status:
```

```
'approved' })}>Approve</button>
```

```
</div>
```

```
);
```

```
}
```

4 Obsidian Sync Scope (Reusable):

```
jsx
```

```
export function useObsidianScope() {
```

```
const { data, setData } = useScope('obsidian', {
  vault: 'CrystalCastle',
  connected: false,
  lastSync: null,
  error: null
});

// Auto-connect on mount
useEffect(() => {
  fetch(`${data.apiUrl}/vault`).then(() => setData({
    connected: true }));
}, [data.apiUrl, setData]);

return data;
}
```

Performance Advantages

 Shallow Diff: Skips re-renders if data is

identical

✅ Isolation: Scopes don't interfere → clean state boundaries

✅ Memoized: All functions/values stable → safe for deps

✅ Auto-Cleanup: No memory leaks (unless persist: true)

✅ Registry-Based: Single source of truth → no duplication

🧩 Folder Structure (Integrated)

plaintext

src/

| — hooks/

| | — useScope.js # ✅ New Scope Hook

| — scopes/ # 📁 Scope Definitions

| | — AppScope.jsx

| | — AuthScope.jsx

| | — DolaScope.jsx

```
| └─ ObsidianScope.jsx
| └─ services/
|   └─ client.js
|   └─ obsidian.js
└─ components/
    └─ PRCard.jsx # Uses Scopes
```



Add to HTML Bundle

Add this inside `<script>` tag in your HTML:

html

```
<script type="module">
```

```
// Paste useScope source above here
```

```
</script>
```

Want me to add TypeScript types, Redux-like

selector support, or persist-to-LocalStorage
directly into useScope ?

